

CMPSC 132 Final Project Report

Eyad Gomaa

Project Description

This project is a terminal-based Number Guessing Game where the computer generates a random number, and the player attempts to guess it. After each guess, the program provides feedback indicating whether the guess is too high, too low, or correct. The game continues until the correct number is guessed or the player runs out of attempts (depending on difficulty level).

Approach and logic used

I designed this program using an object-oriented approach with a main class called “NumberGuessingGame”. The game begins by prompting the player to select a difficulty level, which determines the range the number will be between and the number of attempts the player has.

I used while loops to keep the game running and to continuously prompt the player to guess a number until they correctly guessed the number, ran out of attempts, or chose to quit the game.

Each guess is compared to the randomly generated number, and the player is provided with feedback if their guess is higher or lower than the generated number. The game also keeps track of the number of valid attempts the player makes and displays it when the game ends.

Data structures used

This program uses several important data structures and variables to manage the game effectively. A class structure is used to organize all game-related data and functions together. Within the class, integer variables are used to store the randomly generated number, the number of attempts made by the player, and the maximum number of allowed attempts depending on the selected difficulty.

String variables are used to store the difficulty level and user input values. Boolean variables are also used throughout the program to control loops and track whether the game is over or whether valid input has been entered.

I did my best to keep all the game data organized to make the code easier to read and maintain.

Challenges faced

Although this program was relatively simple to code, one of the more challenging steps was designing the difficulty system because each difficulty level required different number ranges and attempt limits. This was solved by organizing the difficulty setup into a separate function, making the program cleaner and easier to modify later. The other main challenge was validating user input and preventing the program from crashing. Since the input function always returns a string, the program needed to safely convert user input into integers while handling invalid entries such as letters, symbols, or negative numbers. This was solved using try-except statements and additional validation checks.

Possible improvements

One improvement that could be made to the game is specifying the feedback given. For example, if the guess is within +5 or -5 from the correct number, it could return “very close, but too high/low” instead of just “too high/low”. Another addition could be allowing the player to select the range and number of attempts, so that the game is more customizable and entertaining. One other idea I had was to add a scoreboard that displays how many times the player has won the game, as well as their best game.